

Spring Data JPA

>> Spring Data JPA is a part of Spring Data framework that provides simplified data access using JPA (Java Persistence API).

>> It reduces boilerplate code for database operations like CRUD, queries, and transactions.

>> Key Features of Spring Data JPA

- Provides CRUD repository interfaces for database operations.
- Supports JPQL, Native Queries, and Derived Queries.
- Automatic implementation of repository methods.
- Integrates easily with Spring Boot.
- Supports pagination and sorting.
- Handles transactions seamlessly.

Spring Data JPA Setup

>> Maven Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
</dependency>
```

>> Database Config (application.properties)

```
spring.datasource.url=jdbc:mysql://localhost:3306/test
spring.datasource.username=root
spring.datasource.password=1234
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

- spring.datasource.url → Database connection URL
- spring.datasource.username → DB username
- spring.datasource.password → DB password
- spring.jpa.hibernate.ddl-auto=update → Automatically creates/updates tables
- spring.jpa.show-sql=true → Prints SQL queries in console
- spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect → Tells Hibernate to generate SQL optimized for MySQL8

```

-----
--
>> Entity

-- @Entity                → marks class as a database entity.
-- @Table(name="student-table") → maps class to a specific table.
-- @Id                    → marks primary key.
-- @Column                → custom column mapping.
-- @GeneratedValue(strategy=GenerationType.IDENTITY) → auto-generates ID.

```

>> Example

```

@Entity
@Table(name="users")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable=false, unique=true)
    private String email;

    private String name;
}

```

```

-----
--
>> Repository

```

```

-- Spring Data JPA automatically provides JpaRepository, CrudRepository,
PagingAndSortingRepository.

-- JpaRepository          → full CRUD + JPA-specific features.
-- CrudRepository         → basic CRUD operations.
-- PagingAndSortingRepository → CRUD + pagination & sorting.

```

Example

```

public interface UserRepo extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}

```

```

-----
--
>> Query Methods (Derived Queries)

```

```

-- Spring Data automatically creates queries based on method names.

```

Method Name	Query Example
findByName	SELECT u FROM User u WHERE u.name = ?1

```
| findByEmailAndName    | SELECT u FROM User u WHERE u.email = ?1 AND u.name = ?2
| findByAgeGreaterThan | SELECT u FROM User u WHERE u.age > ?1
| existsByEmail         | Checks if a record exists
| countByStatus         | Returns count based on condition
|
```

```
-----
--
```

>> JPQL Queries (@Query)

-- It looks like SQL, but it does not work directly on database tables, it works on Entities and their fields.

-- Advantage → If the database changes, JPQL queries still work because they depend on the Entity structure, not the table structure.

Example

```
@Entity
@Table(name="users")
public class User {
    @Id
    private Long id;

    private String name;
    private String email;
}
```

JPQL Query

```
@Query("SELECT u FROM User u WHERE u.email = ?1")
User findUserByEmail(String email);
```

Explanation

```
-- User → Entity class
-- u → Entity ka alias (like table ka short name)
-- email → Entity field ka naam, table column ka nahi
-- ?1 → Positional parameter (first parameter method me)
```

>> Positional Parameters

```
-- JPQL me ?1, ?2, ?3 positional parameters hote hain
-- Method me jo argument pass karte hain, wo order ke hisaab se replace hota hai
```

```
-- Named Parameters : Positional parameters ke bajaye named parameters use kar sakte hain ( Syntax: : paramName )
```

```
-----
--
```

>> Native Queries

```
-- Executes raw SQL queries on database.
-- Use nativeQuery=true.
```

```

@Query(value="SELECT * FROM users WHERE email=?1", nativeQuery=true)
User findByEmailNative(String email);

-----
--

>> Relationships in JPA

-- OneToOne

@OneToOne(cascade=CascadeType.ALL)
@JoinColumn(name="profile_id")
private Profile profile;

=====

-- OneToMany / ManyToOne

@OneToMany(mappedBy="department", cascade=CascadeType.ALL, fetch=FetchType.LAZY)
private List<Employee> employees;

@ManyToOne
@JoinColumn(name="department_id")
private Department department;

=====

-- ManyToMany

@ManyToMany(cascade=CascadeType.ALL)
@JoinTable(
    name="student_course",
    joinColumns=@JoinColumn(name="student_id"),
    inverseJoinColumns=@JoinColumn(name="course_id")
)
private List<Course> courses;

-----
--

>> Fetch Types (already discussed in Spring Boot)

>> Cascade Types (already discussed in Spring Boot)

>> Pagination and Sorting (already discussed in Spring Boot)

>> Transactional (already discussed in Spring Boot)

=====

>> Auditing

-- Automatic fields like createdAt and updatedAt.
-- Use @EnableJpaAuditing and annotations.

@CreatedDate
private LocalDateTime createdAt;

@LastModifiedDate
private LocalDateTime updatedAt;

-----
--

```

>> Specifications / Criteria API (Advanced Queries)

-- Use Criteria API for dynamic queries.
-- Example : Specification<User> to filter based on conditions dynamically.

```
public static Specification<User> hasEmail(String email) {  
    return (root, query, cb) -> cb.equal(root.get("email"), email);  
}
```

--

>> Projections / DTO Mapping

-- Fetch only required fields instead of whole entity.
-- Use interface-based projections or DTOs with JPQL.

```
@Query("SELECT new com.app.dto.UserDTO(u.name, u.email) FROM User u")  
List<UserDTO> fetchUsersDTO();
```

--

>> Locking

-- Avoid concurrent modification issues.
-- Types : Optimistic Locking, Pessimistic Locking

--

```
@Version  
private Integer version; // optimistic locking
```

--

>> Integrating with Spring Boot

-- Repository layer automatically scanned → dependency injection in Service layer.
-- Works seamlessly with Spring Security, Validation, Transactions, and REST APIs.

--

>> Spring Data JPA → Makes working with the database very easy.

>> CRUD, relationships, queries, transactions → all mostly automatic.

>> The entities and repositories you create → Spring Boot automatically scans them and injects them into the Service layer.

--